

Lab Assignment: Approximate String Matching

Objective

In this lab you will build the standard tools for finding "almost equal" strings, which power typo correction, record deduplication and entity matching. You will implement each from scratch, measure it, and compare it with alternatives.

- **Format:** Python 3, C/C++
- **Rules:** Parts 1–3 must be your own code, with no edit-distance libraries. A library is allowed for Jaro-Winkler in Part 4.
- **Submit:** one code file (or notebook) plus a one-page report, at the end of the session.

Tasks

Part	What to do	Checkpoint
0. Read basics	Fill the DP table by hand for kitten → sitting.	Distance is 3
1. Levenshtein DP	Write <code>lev_full(a, b)</code> using the $O(mn)$ table, then <code>lev_two_row(a, b)</code> using two rows. Test empty strings and 1,000 random pairs.	Both versions agree on every pair
2. Sellers variant	Set the first row to all zeros and take the minimum of the last row. Find every end position where "algorithm" matches a paragraph you choose with at most 2 edits.	Exact matches score 0
3. BK-tree	Build a BK-tree on a word list of 10,000 or more words. Implement <code>query(word, k)</code> , pruning children whose edge label is outside $d-k$ to $d+k$. For $k = 1, 2$,	Same results as brute force

Part	What to do	Checkpoint
	3 and 100 misspelled queries, count distance computations against brute force.	
4. Comparison	Score at least 6 pairs (a name variant, a typo, reordered words, an abbreviation) with Levenshtein, Jaro-Winkler and bigram Jaccard.	Table of all three scores

Report

Your one-page report contains the BK-tree measurements table ($k \times$ brute-force vs BK-tree distance computations), the Part 4 comparison table, and short answers to three questions:

1. Why can the two-row version return the distance but not the edit script?
2. Why does the Sellers change turn whole-string comparison into approximate substring search?
3. Why does the triangle inequality allow BK-tree pruning, and why does the saving shrink as k grows?